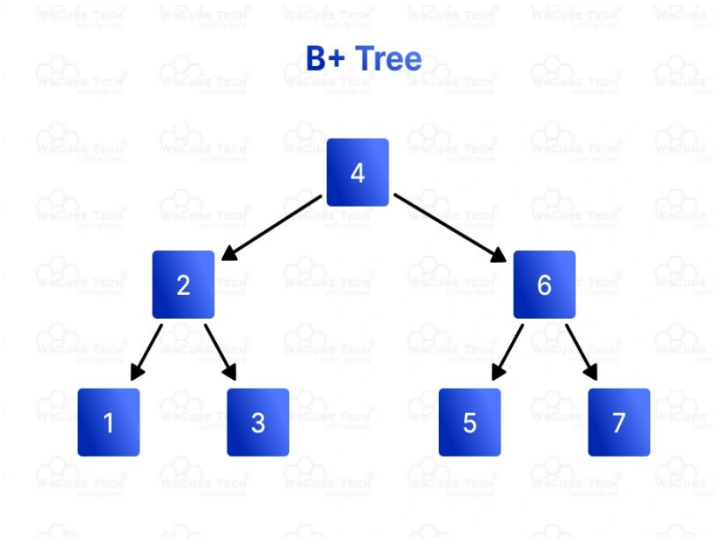
B -Trees

Introduction

A B-Tree in data structure is a self-balancing tree used to efficiently store and manage large datasets. In a B-Tree, each node can have multiple children, ensuring the tree remains balanced and operations like search, insertion, and deletion are performed in O(log n) time. Let's learn everything about B-Tree data structure, including its definition, examples, properties, operations, applications, implementation, and more.

What is B-Tree?

A B-Tree is a special kind of data structure that helps store and manage large amounts of data efficiently. It is called a "tree" because it looks like an upside-down tree with branches. Each part of the tree is called a node, and each node can have many children. This helps keep the tree balanced, meaning it doesn't get too tall and narrow or too wide and short



In a B-Tree:

- Each node can hold multiple values.
- Each node can have multiple children.
- The tree stays balanced by making sure all the leaves (the nodes at the bottom) are at the same level.

The main idea of a B-Tree is to keep data organized in a way that makes it quick to find, add, or remove information. This is especially important when working with large amounts of data, like in databases or file systems.

Properties of B-Tree

The following are the key properties of B-Tree in data structure:

Balanced Tree

B-Trees are balanced, meaning that all the leaf nodes (the nodes at the bottom of the tree) are at the same level. This ensures that the tree remains short and wide, which helps in efficient data retrieval.

Node Structure

Each node in a B-Tree can hold multiple keys (values) and children. The number of keys a node can hold is determined by the order of the B-Tree, denoted as 'm'.

Order 'm'

The order 'm' of a B-Tree defines the range of keys and children each node can have:

- A node can have at most 'm' children.
- A node can have at most 'm-1' keys.

Each node, except the root, must have at least ⌈m/2⌉ children and ⌈m/2⌉-1 keys. The root node must have at least two children if it is not a leaf node.

Keys in Nodes

The keys within a node are stored in a sorted manner. This helps in efficient searching within the node.

Child Pointers

Each node has pointers to its child nodes. The number of child pointers is always one more than the number of keys in the node.

Data Range in Subtrees

For any given node:

- All keys in the left subtree are smaller than the keys in the node.
- All keys in the right subtree are greater than the keys in the node.

Height of the Tree

The height of a B-Tree is kept low by allowing nodes to have multiple children. This ensures that operations like search, insertion, and deletion can be performed efficiently.

Operations on B-Tree

B-Tree in data structure support several fundamental operations, including insertion, deletion, and searching:

1. Insertion

Insertion in B-Tree involves adding a new key into the appropriate position while maintaining the tree's properties. If a node becomes full, it is split into two nodes, and the middle key is promoted to the parent node.

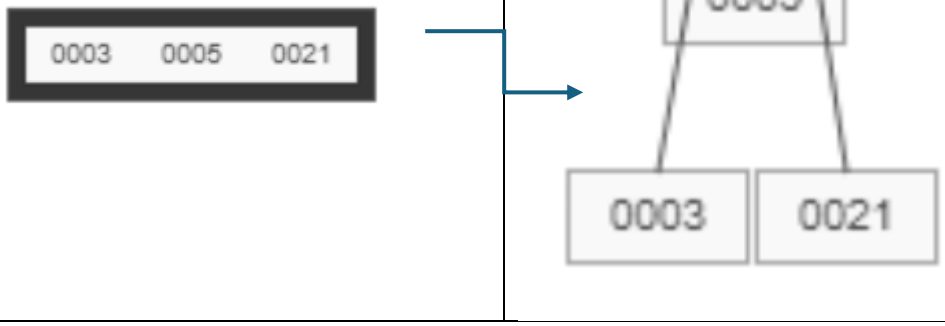
Algorithm:

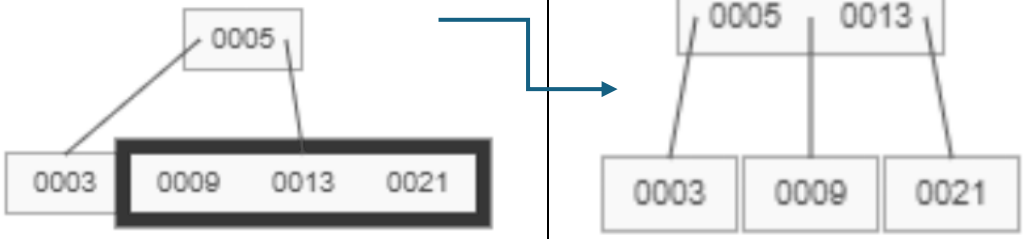
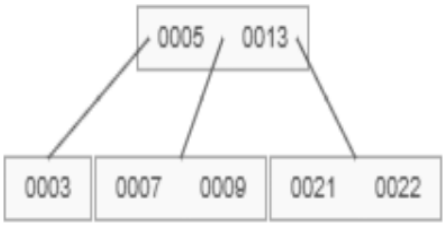
- Search for the correct leaf node to insert the new key.
- Insert the key into the leaf node in sorted order.
- If the leaf node has more keys than allowed (m-1 keys), split the node:

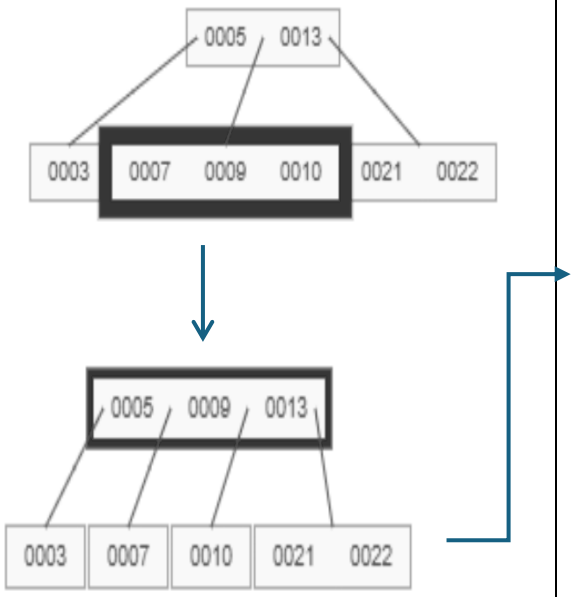
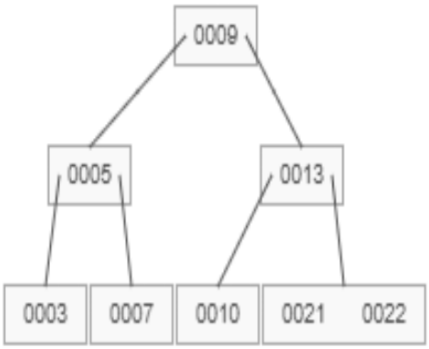
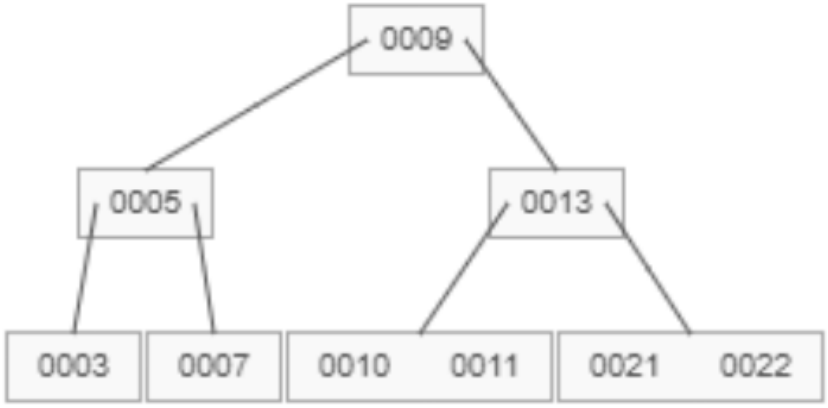
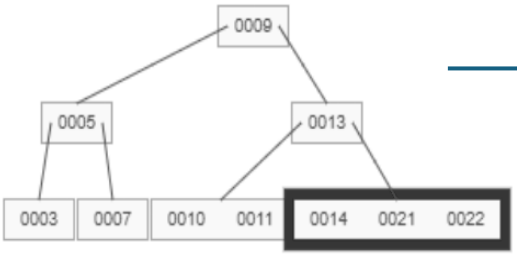
- Find the median key in the node.
- Create a new node and move half of the keys to the new node.
- Promote the median key to the parent node.
- If the parent node also becomes full, repeat the splitting process recursively up to the root.
- If the root becomes full, create a new root and split the old root into two children.

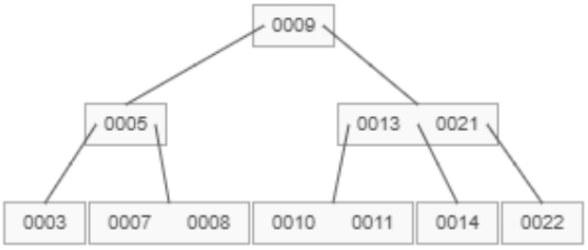
Example:

Create B Tree of the Following Elements 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 of order M=4

Insert	Tree Representations
5	
3	
21	
9	

	 0005 0003 0009 0021
13	 0005 0003 0009 0013 0021 0005 0013 0003 0009 0021
22	 0005 0013 0003 0009 0021 0022
7	 0005 0013 0003 0007 0009 0021 0022

10	 0005 0013 0003 0007 0009 0010 0021 0022 0005 0009 0013 0003 0007 0010 0021 0022	 0009 0005 0013 0003 0007 0010 0021 0022
11	 0009 0005 0013 0003 0007 0010 0011 0021 0022	
14	 0009 0005 0013 0003 0007 0010 0011 0014 0021 0022	 0009 0005 0013 0021 0003 0007 0010 0011 0014 0022
8		

	
16	

2. Deletion

Deletion in a B-Tree involves removing a key from the appropriate position while maintaining the tree's properties. This may require borrowing a key from a sibling or merging nodes to maintain the minimum degree property.

Algorithm:

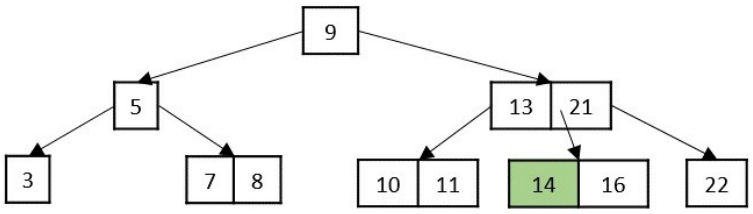
- Search for the key to be deleted.
- If the key is found in a leaf node, remove it directly.
- If the key is found in an internal node, there are two cases:
- Replace the key with the predecessor (maximum key in the left subtree) or successor (minimum key in the right subtree) and recursively delete the predecessor/successor.
- If the key is not found in the node but exists in the subtree, recursively delete it from the appropriate subtree.
- If a child node has fewer keys than the minimum required, borrow a key from a sibling or merge with a sibling to maintain the minimum degree property.

Example:

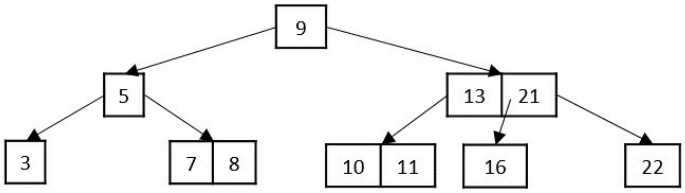
The deletion operation in a B tree is slightly different from the deletion operation of a Binary Search Tree. The procedure to delete a node from a B tree is as follows −

Case 1 − If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.

Delete key 14

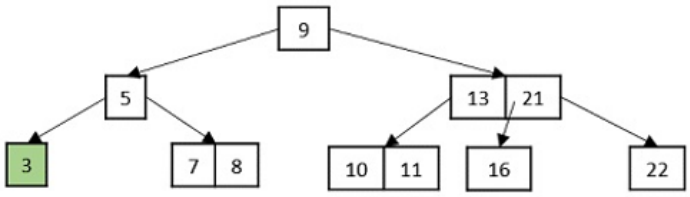


Delete key 14

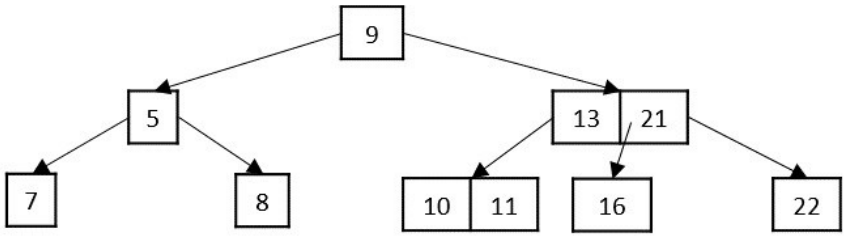


Case 2 – If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling. In case if both siblings have exact minimum number of keys, merge the node in either of them.

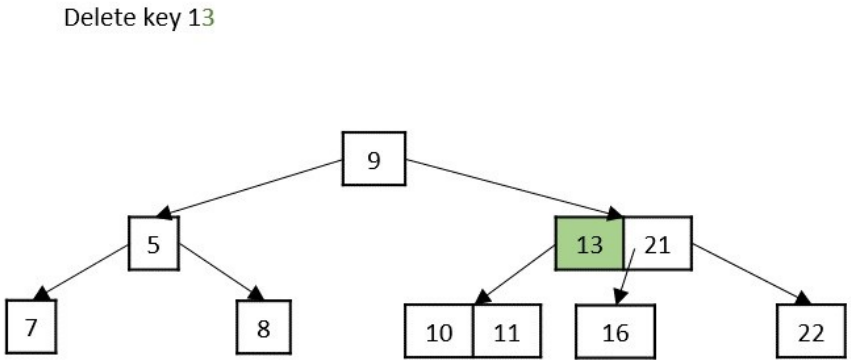
Delete key 3

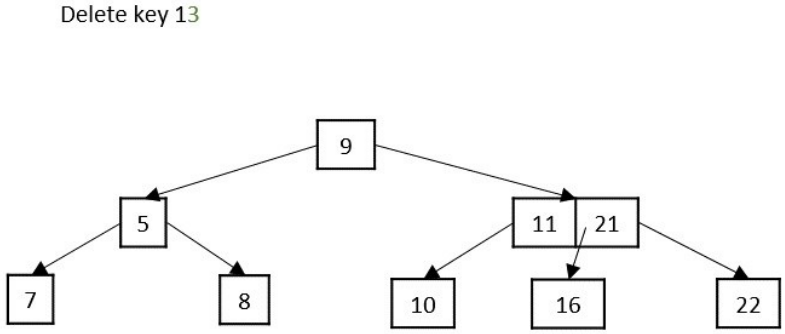


Delete key 3



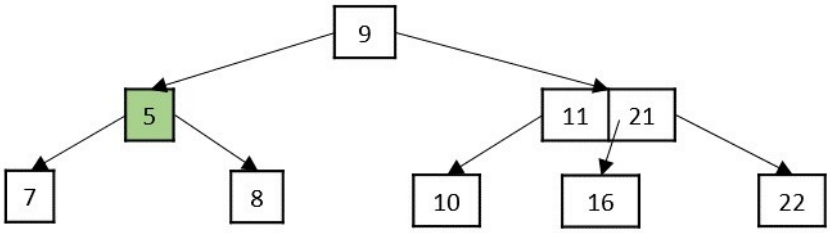
Case 3 − **If the key to be deleted is in an internal node, it is replaced by a key in either left child or right child based on which child has more keys. But if both child nodes have minimum number of keys, theyre merged together.**



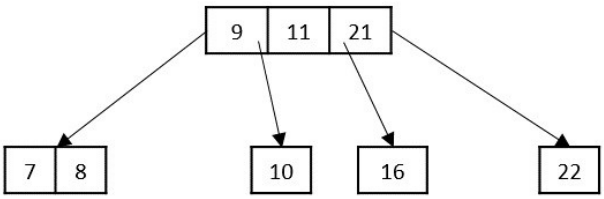


Case 4 − If the key to be deleted is in an internal node violating the minimum keys property, and both its children and sibling have minimum number of keys, merge the children. Then merge its sibling with its parent.

Delete key 5



Delete key 5



3. Searching

Searching in a B-Tree involves finding a key within the nodes. The search process is similar to binary search and takes advantage of the sorted order of keys in each node.

Algorithm:

- Start at the root node and check if the key is in the current node.
- If the key is found, return the node and the index of the key.
- If the key is not found and the node is a leaf, return None (the key does not exist in the tree).
- If the key is not found and the node is not a leaf, recurse into the appropriate child node:
- Compare the key with the keys in the current node to determine the correct child to search.

Example:

Consider the B-Tree after inserting 10, 20, 5, 6, 12.

Search for the key 12:

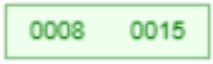
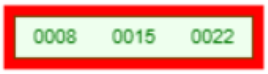
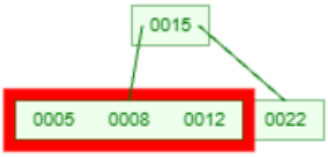
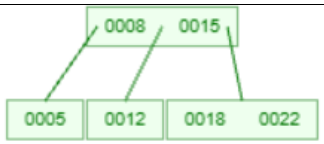
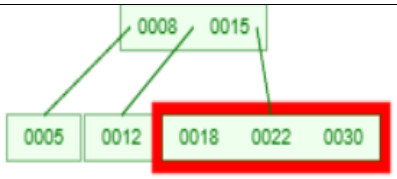
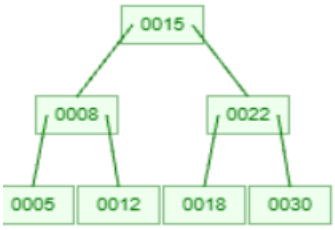
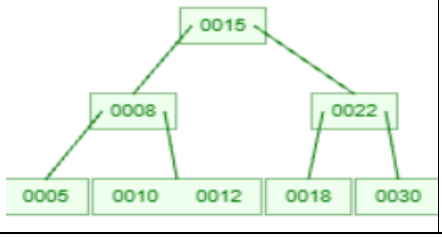
- Start at the root [10].
- Key 12 is greater than 10, move to the right child [12, 20].
- Key 12 is found in the node [12, 20].

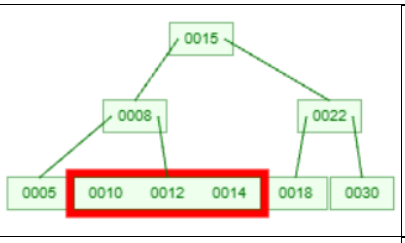
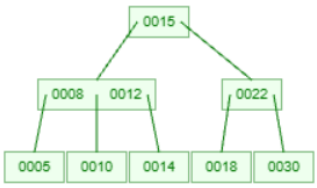
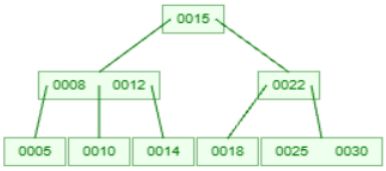
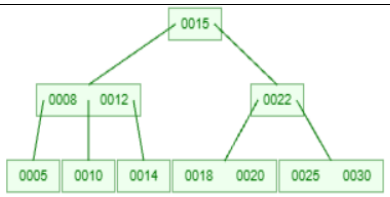
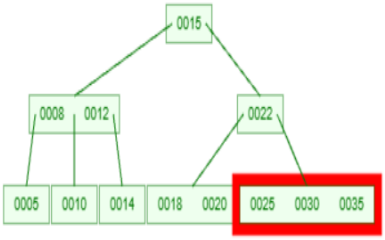
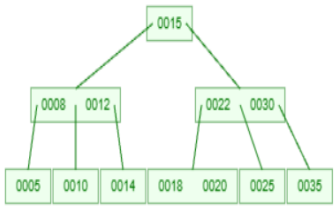
B-TREE PROBLEMS

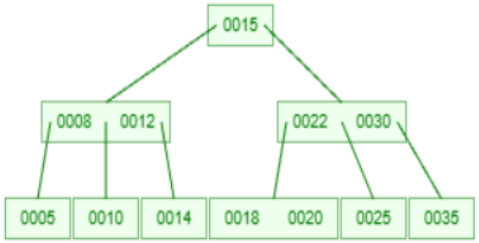
Question 1

Construct a B-Tree of order 4 by inserting the keys 15, 8, 22, 5, 12, 18, 30, 10, 14, 25 in the given order. Show node splitting, key promotion, and height changes after each insertion. After construction, insert 20 and 35, then delete 8 and 22. Show all redistribution or merging steps during deletion.

Insertion:

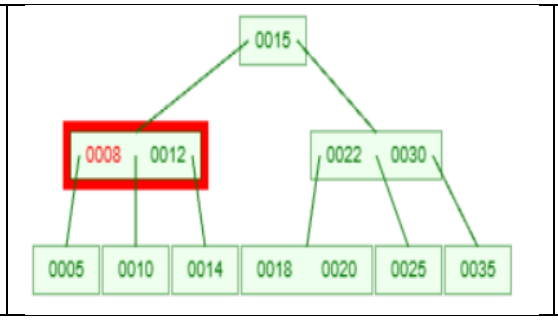
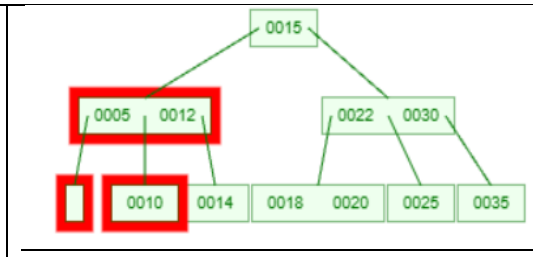
Insert 15			
Insert 8			
Insert 22			
Insert 5			
Insert 12			
Insert 18			
Insert 30			
Insert 10			

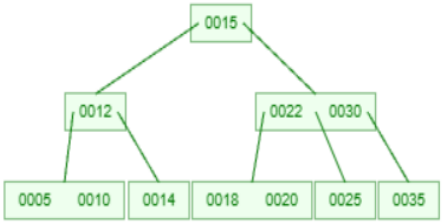
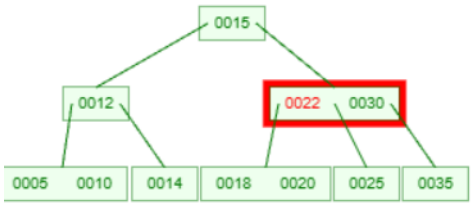
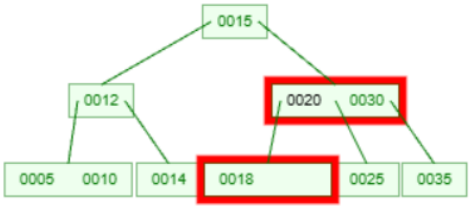
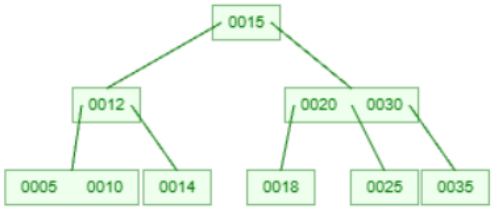
Insert 14			
Insert 25			
Insert 20			
Insert 35			



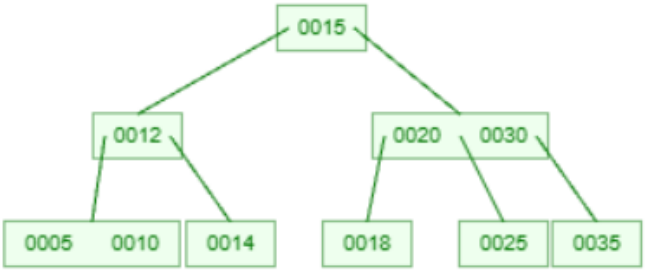
Tree after Insertion:

Deletion:

Delete 8		

		
Delete 22		 Step 3: (below) 

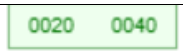
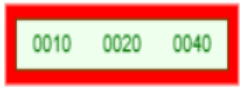
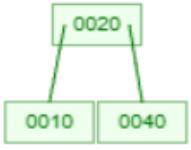
Final B-Tree:

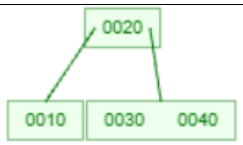
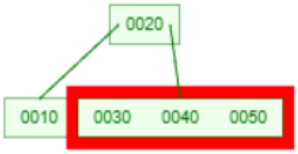
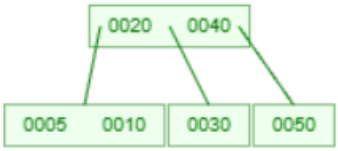
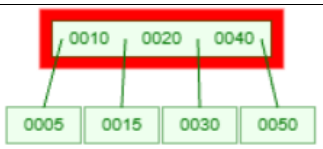
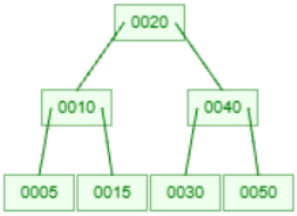
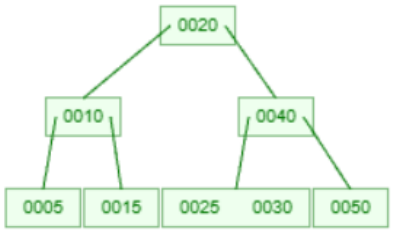
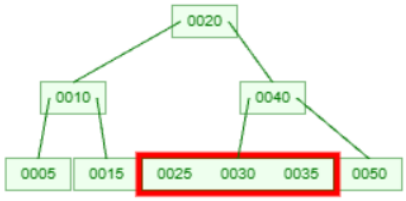
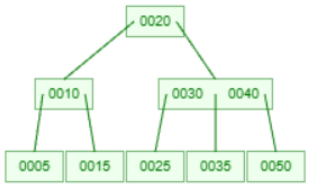
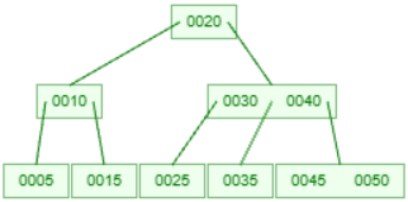
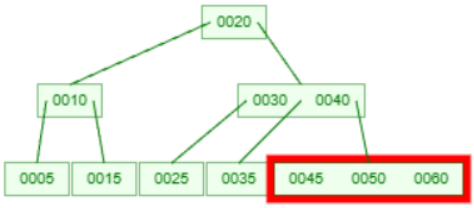
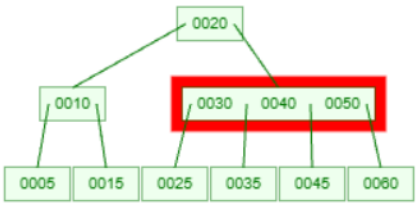
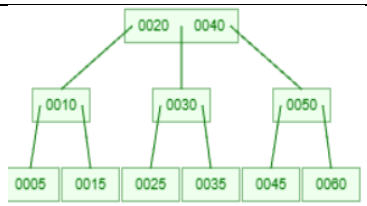


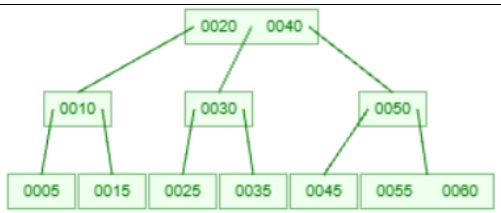
Question 2:

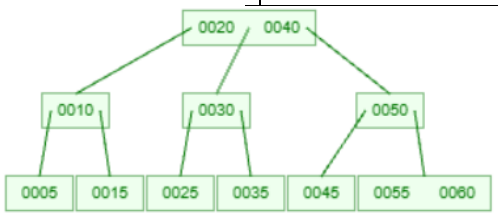
Construct a B-Tree of order 4 by inserting the keys 20, 40, 10, 30, 50, 5, 15, 25, 35, 45 in the given order. Illustrate node splits, promoted keys, and height variations. After forming the tree, insert 60 and 55, then delete 10 and 30.Show all steps involved in deletion using redistribution or merging.

Insertion:

20			
40			
10			

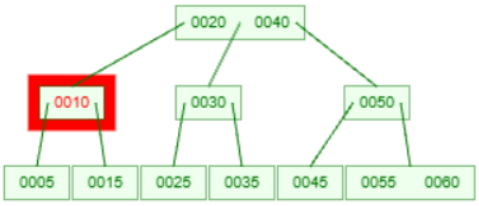
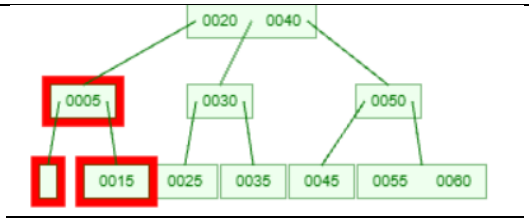
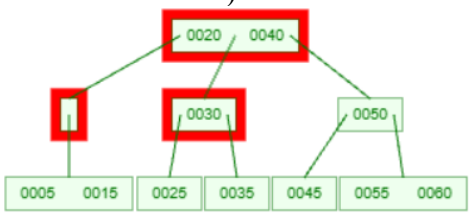
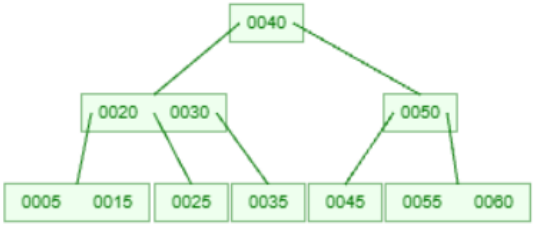
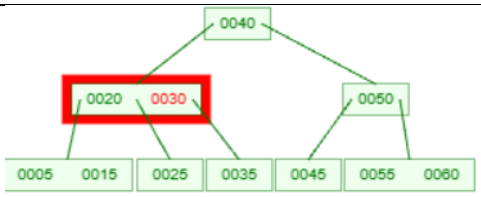
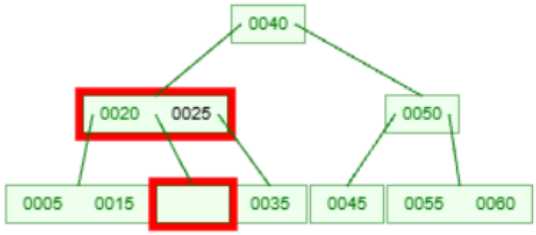
30			
50			
5			
15			
25			
35			
45			
60			

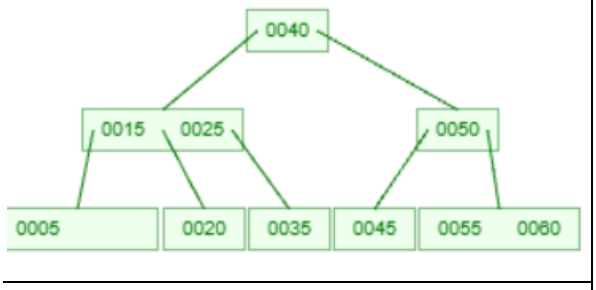
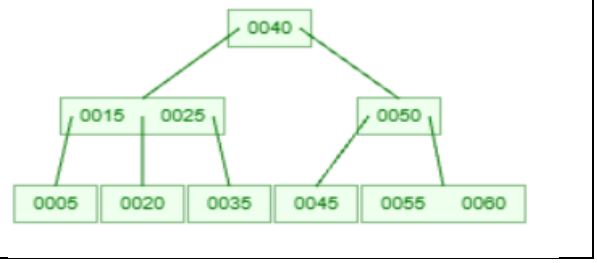
55			

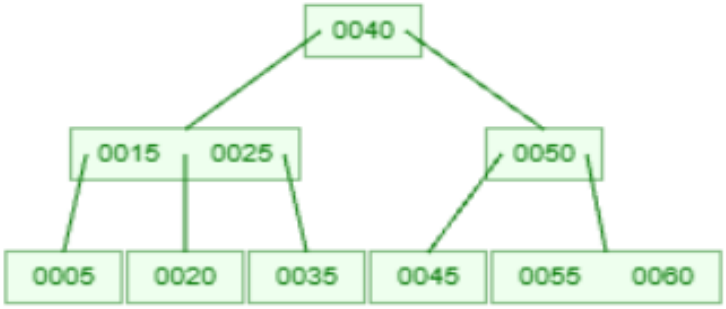


Tree after Insertion:

Deletion:

Delete 10		 (continuation below)  
Delete 30		 (continuation below)

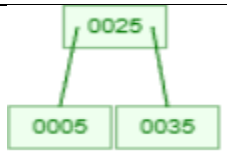
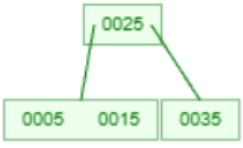
		 

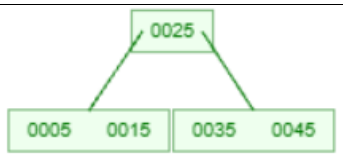
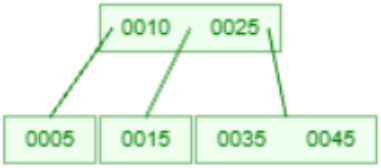
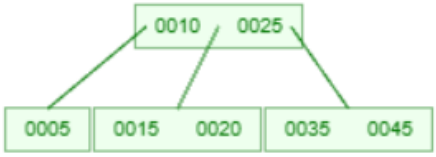
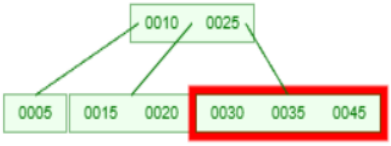
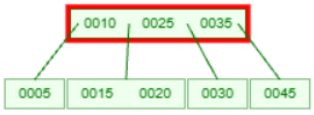
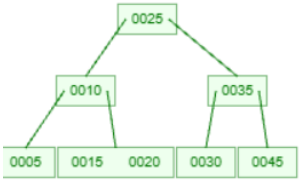
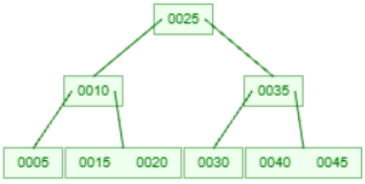
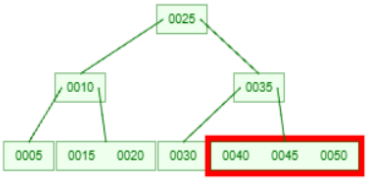
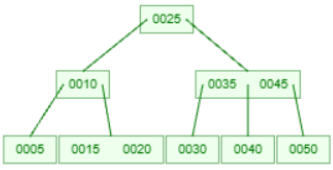
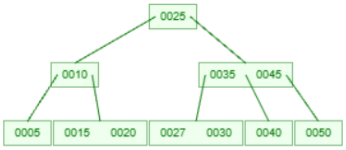
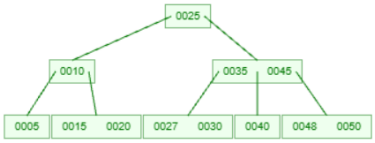


Final B-Tree:

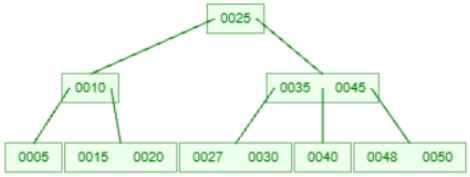
Question 3

Construct a B-Tree of order 4 by inserting the keys 25, 5, 35, 15, 45, 10, 20, 30, 40, 50 in the given order. Show node splitting, promotion of middle keys, and tree height changes. After construction, insert 27 and 48, then delete 5 and 35. Explain redistribution and merging during deletion.

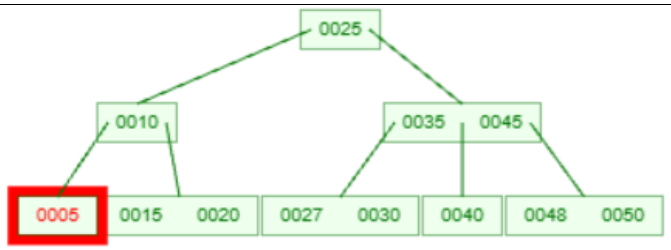
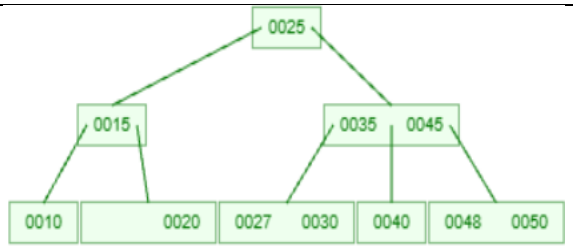
25			
5			
35			
15			

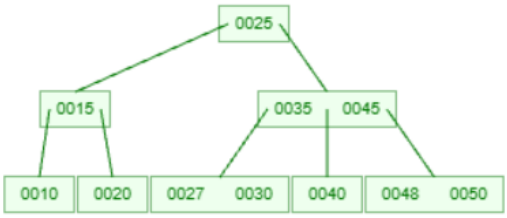
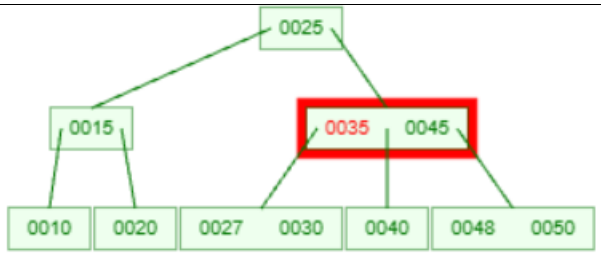
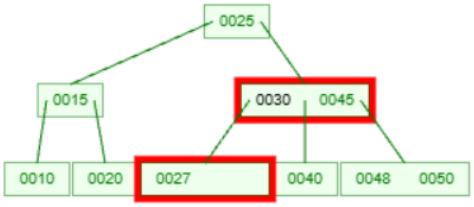
45			
10			
20			
30			
40			
50			
27			
48			

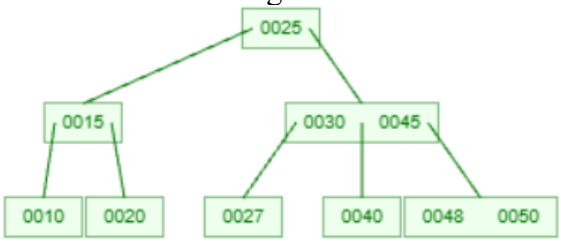
Tree after Insertion:

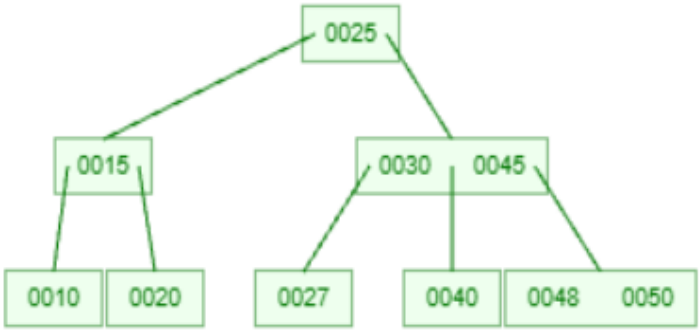


Deletion:

<table>
<tr>
<td>Delete 5</td>
<td> 
</td>
<td> 

Final tree after deleting 5

</td>
</tr>
<tr>
<td>Delete 35</td>
<td> 
</td>
<td> 

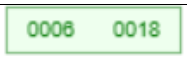
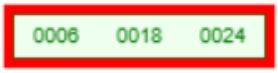
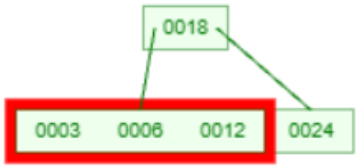
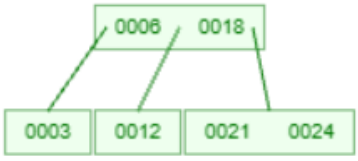
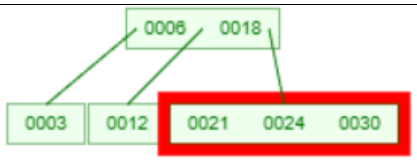
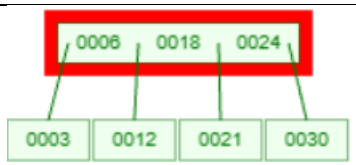
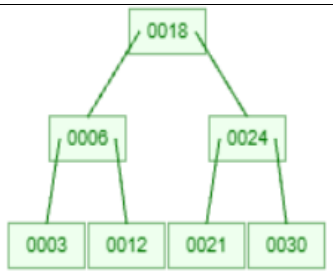
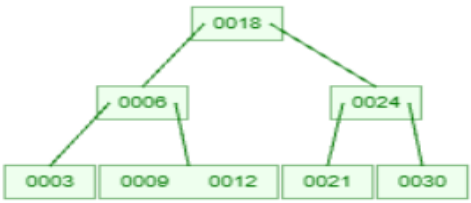
Final tree after deleting 35

</td>
</tr>
</table>

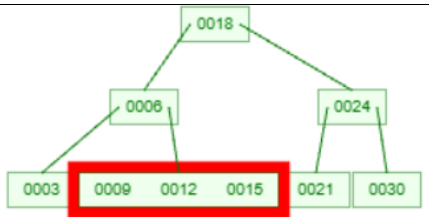
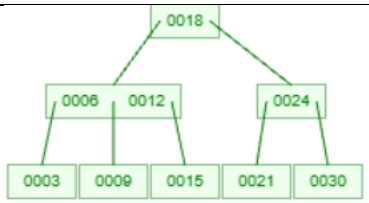
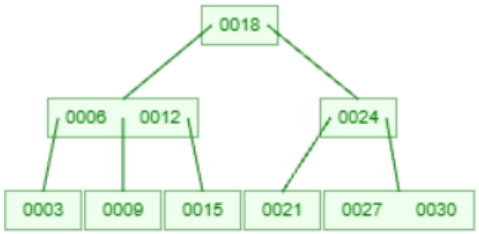
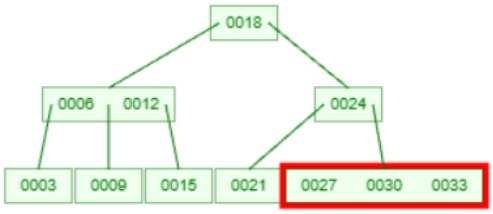
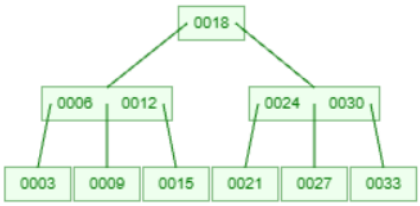
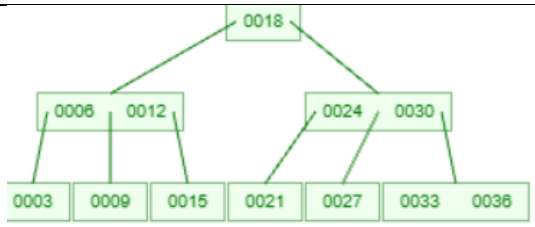


Final B-Tree:

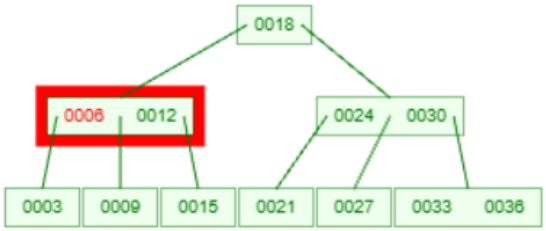
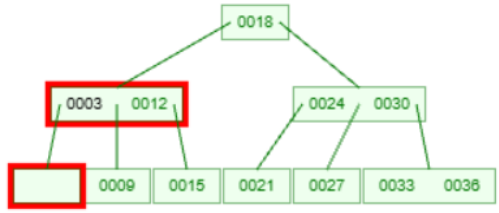
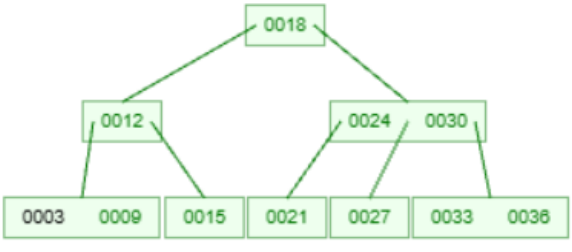
Question 4

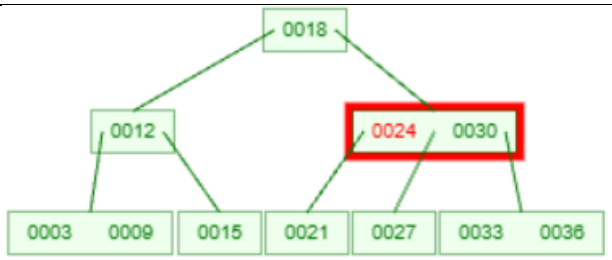
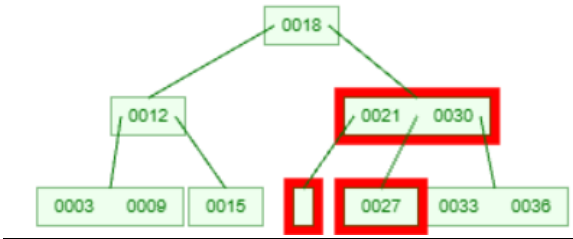
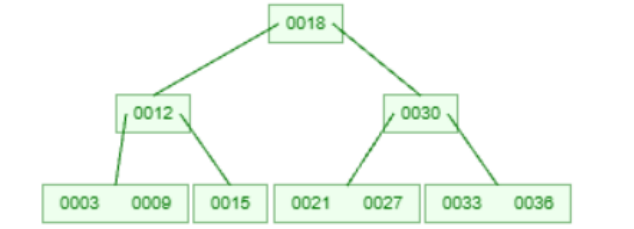
Construct a B-Tree of order 4 by inserting the keys 18, 6, 24, 3, 12, 21, 30, 9, 15, 27 in the given order. Show the B-Tree after each insertion with splits and promotions.After obtaining the final tree, insert 33 and 36, then delete 6 and 24.Show all deletion steps involving redistribution or node merging.

18			
6			
24			
3			
12			
21			
30			
9			

15			
27			
33			
36			

Deletion:

Delete 6		 Continuation: 

Delete 24		 Continuation:  

Final B-Tree: